Requirement on academic honesty (COMP2090SEF):

1. Each group must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP2090SEF
Data Structures, Algorithms And Problem Solving

Course Project Report

Group

Submission date:

We declare that:
(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;
(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
Laiyutao	13996440
Longenze	14018180
CheungWancen	14002864

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

3. All members are liable to the plagiarism incidence, even if a free-rider.

1. Principle of Operation: How Radix Sort Works

Radix Sort is a non-comparative integer sorting algorithm. Instead of comparing elements directly against one another, it sorts data by processing individual digits of the numbers sharing the same significant position and value.

The underlying principle relies on two core concepts: **digit-by-digit processing** and **stable sub-sorting**.

- **Least Significant Digit (LSD) Approach:** The algorithm processes the numbers starting from the Least Significant Digit (the rightmost digit, or units place) and progresses leftwards to the Most Significant Digit (the leftmost digit).
- **Bucket Distribution (Stable Sub-sort):** For each digit position, the algorithm uses a stable sorting sub-routine. In a base-10 number system, it creates 10 "buckets" representing the digits 0 through 9. During a single pass, every number in the array is distributed into the bucket corresponding to its current digit's value.
- **Sequential Collection:** After all numbers are distributed for the current digit, they are collected out of the buckets in sequential order (from bucket 0 to bucket 9) and placed back into the main array.
- **Preserving Relative Order (Stability):** Because the collection process is stable, elements that share the exact same digit in the current pass maintain the relative order they established in previous passes (which sorted their less significant digits). By the time the algorithm finishes processing the most significant digit of the largest number, the entire array is globally sorted.

2. Time Complexity Analysis

The time complexity of Radix Sort is not determined by element comparisons, but strictly by the number of elements, the base of the numbering system, and the length of the largest number.

Let:

- n = the number of elements in the array
- d = the maximum number of digits in the largest element
- k = the base of the number system (for our decimal matrix, k=10)

Analysis per pass: During each iteration (i.e., for a single digit position), the algorithm performs two main operations:

1. Iterating through all n elements to extract the target digit and distribute them into the k buckets. This takes $O(n)$ time.
2. Iterating through the k buckets to collect the elements back into the main array list. This takes $O(k)$ time. Thus, a single pass takes $O(n+k)$ time.

Total Time Complexity: The algorithm must repeat this single-pass operation for every digit position in the largest number. Therefore, the distribution and collection phase executes exactly dd times.

Overall Time Complexity:

Time Complexity=$O(d.(n+k))$

- **Best, Average, and Worst Case:** $O(d.(n+k))$ Radix Sort does not possess a "short- circuit" mechanism for already sorted arrays. It will consistently execute

exactly dd passes across all n elements, making its best, average, and worst-case time complexities identical.

- **Practical Implications:** Because the base k is usually a small constant like 10, and the maximum number of digits dd is also small relative to n in many datasets, the time complexity simplifies to near linear time: O(n). This gives Radix Sort a significant theoretical advantage over comparative sorting algorithms like Merge Sort or Quick Sort (O($n$log n)) when handling massive datasets of comparable integers.

Implement

（There is related code in the code file ）

Application example 1: Massive Integer Dataset Sorting for Big Data Analytics

In big data platforms, log analysis, user behavior tracking, and financial transaction systems often generate hundreds of millions to billions of integer-format records (e.g., user IDs, transaction serial numbers, timestamps). These integers have fixed digit lengths and require fast, stable sorting for subsequent statistics and querying.

Process

1. Extract integer fields (user ID/transaction number) from the dataset.
2. Use LSD Radix Sort to sort from least significant digit to most significant digit.
3. Distribute elements into 10 buckets by current digit, collect sequentially, and repeat for all digit positions.
4. Output sorted dataset for data analysis and query acceleration.

Application example2: Multi-keyword Sorting for Electronic Document Indexing

Office systems, library management, and electronic archive platforms need to sort files/documents by multi-digit numbering (e.g., document number: 20260324001, book ISBN: 9787111642669). Multi-keyword stable sorting is required to ensure correct hierarchical order.

Process

1. Parse document numbers into independent digit positions (year, month, day, serial number).
2. Apply Radix Sort from the lowest digit (serial number) to the highest digit (year).
3. Bucket distribution and sequential collection maintain stable order.
4. Generate sorted document index list for quick retrieval and management.

Basic Principle of Matrix

A matrix is a 2D data structure for storing data in rows and columns. Each element can be found only by its row and column index. In Python, it is a nested list: the outer list for rows, inner lists for column values. It is an extension of 1D arrays, used for structured tabular data storage and operations.

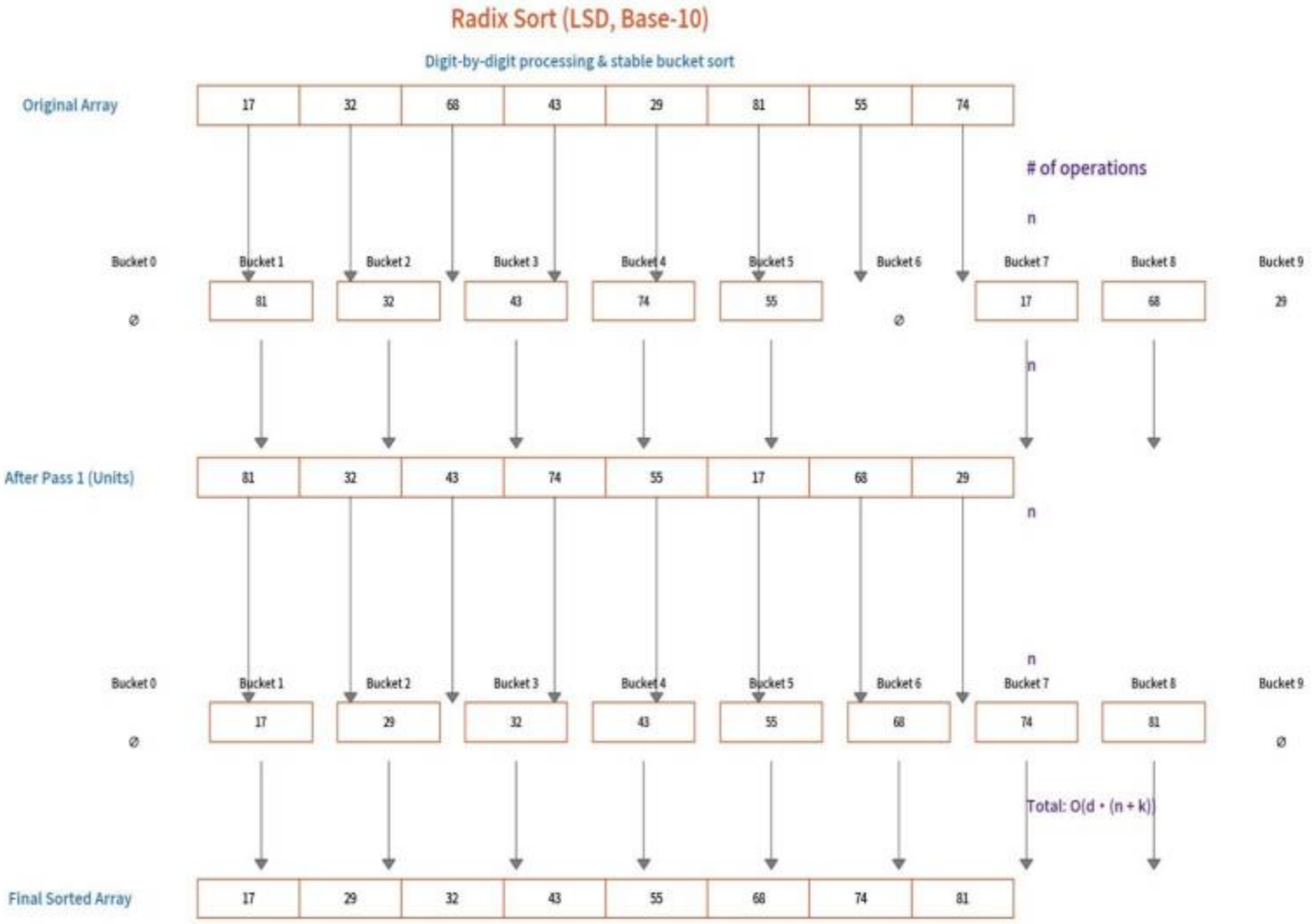
Main Method

In this code, the matrix is a non-empty 2D integer list, mainly to sort by a specified column. Key steps: extract the target column as the sort key with list comprehension; use a boolean list to mark row usage and solve reordering problems from duplicate column values; match unmarked original rows with sorted column values and reorder into a new matrix; access and print elements with nested loops; use the custom radix sort to reorder the whole matrix by the sorted column.

Development

This code is a simple implementation for non-negative integer matrix operations and custom sorting, and it can be further optimized. Expand basic functions (add/delete rows/columns, transpose matrix, modify elements, check column index validity, etc.). Optimize sorting by binding rows with target column values, sort the pairs directly with radix sort to cut time complexity and remove the marking list. Adjust radix sort to support floats and strings (instead of only non-negative integers); use multiple column values as keys for multi-column sorting; apply in-place sorting to modify the original matrix directly and save memory for large matrices.
This matrix (2D nested list) structure has two main practical uses:

Implement
Tabular Data Storage & Analysis
It can store structured data simply, like student grades, inventory lists, or survey data. Each row stands for one item, and each column stands for one feature. This makes it easy to find and change single data points.

Basic Data Structure for Programs
It is widely used in many computing tasks, such as image processing, map building, and basic math calculations. Its row-and-column structure helps us read and process multi-dimensional data easily in Python.

Appendix　Flow Chart of Radix Sort